

Core Issue 1776: Replacement of class objects containing reference members

ISO/IEC JTC1 SC22 WG21

P0137R0

Richard Smith

richard@metafoo.co.uk

2015-09-27

And also Core Issue 1116: Aliasing of union members

☐ Hide deleted text

Change in 1.8 (intro.object) paragraph 1:

The constructs in a C++ program create, destroy, refer to, access, and manipulate objects. **An object is created by a definition (3.1), by a new-expression (5.3.4) or by the implementation (5.18, 12.2) when needed.** An object ~~is~~ **object occupies** a region of storage in its period of construction (12.7 class.ctor), throughout its lifetime (3.8 basic.life), and in its period of destruction (12.7 class.ctor). [Note: A function is not an object, regardless of whether or not it occupies storage in the way that objects do. — end note] ~~An object is created by a definition (3.1), by a new-expression (5.3.4) or by the implementation (12.2) when needed. [...]~~

Drafting note: this maintains the status quo that malloc alone is not sufficient to create an object. Drafting note: the terms "period of construction" and "period of destruction" are in core issue 1517, which is not yet approved, but their use here seems sufficiently clear with or without that change.

Change in 1.8 (intro.object) paragraph 6:

Unless an object is a bit-field or a base class subobject of zero size, the address of that object is the address of the first byte it occupies. Two objects **with overlapping lifetimes** that are not bit-fields may have the same address if one is a subobject of the other, or if at least one is a base class subobject of zero size and they are of different types; otherwise, they shall have distinct addresses. [Footnote: ...] [Example: ...]

Add a new paragraph after 3.7 (basic.stc) paragraph 3:

When the end of the duration of a region of storage is reached, the values of all pointers representing the address of any part of the deallocated storage become invalid pointer values (basic.compound). Indirection through an invalid pointer value and passing an invalid pointer value to a deallocation function have undefined behavior. Any other use of an invalid pointer value has implementation-defined behavior. [Footnote: Some implementations might define that copying an invalid pointer value causes a system-generated runtime fault. — end footnote]

Drafting note: this should apply to all storage durations that can end, not just to dynamic storage duration. On an implementation supporting threads or segmented stacks, thread and automatic storage may behave in the same way that dynamic storage does.

Change in 3.7.4.1 (basic.stc.dynamic.allocation) paragraph 2:

[...] Furthermore, for the library allocation functions in 18.6.1.1 and 18.6.1.2, p0 shall ~~point to~~ **represent the address of** a block of storage disjoint from the storage for any other object accessible to the caller. The effect of indirecting through a pointer returned as a request for zero size is undefined.

Change in 3.7.4.1 (basic.stc.dynamic.allocation) paragraph 2:

If the argument given to a deallocation function in the standard library is a pointer that is not the null pointer value (4.10), the deallocation function shall deallocate the storage referenced by the pointer, **ending the duration of the region of storage.** ~~rendering invalid all pointers referring to any part of the deallocated storage. Indirection through an invalid pointer value and passing an invalid pointer value to a deallocation function have undefined behavior. Any other use of an invalid pointer value has implementation-defined behavior. [Footnote: Some implementations might define that copying an invalid pointer value causes a system-generated runtime fault. — end footnote]~~

Change in 3.8 (basic.life) paragraph 1:

The *lifetime* of an object is a runtime property of the object. An object is said to have *non-vacuous initialization* if it is of a class or aggregate type and it or one of its members is initialized by a constructor other than a trivial default constructor. [Note: initialization by a trivial copy/move constructor is non-vacuous initialization. — end note] The lifetime of an object of type τ begins when:

- storage with the proper alignment and size for type T is obtained, and
- **if the object is a union member or subobject thereof, that union member is the initialized member in the union (8.5.1, 12.6.2), and**

- if the object has non-vacuous initialization, its initialization is complete.

The lifetime of an object generated by the implementation to hold a union member may also begin as described in 5.2.5. [...]

Change in 3.8 (basic.life) paragraph 5:

Before the lifetime of an object has started but after the storage which the object will occupy has been allocated [Footnote: ...] or, after the lifetime of an object has ended and before the storage which the object occupied is reused or released, any pointer that ~~refers to~~ **represents the address of** the storage location where the object will be or was located may be used but only in limited ways. [...]

Change in 3.8 (basic.life) paragraph 7:

If, after the lifetime of an object has ended and before the storage which the object occupied is reused or released, a new object is created at the storage location which the original object occupied, a pointer that pointed to the original object, a reference that referred to the original object, or the name of the original object will automatically refer to the new object and, once the lifetime of the new object has started, can be used to manipulate the new object, if:

- [...]

[Example: ... — end example] **[Note: If these conditions are not met, a pointer to the new object can be obtained from a pointer that represents the address of its storage by calling `std::launder` (18.6 [support.dynamic]). — end note]**

Change in 3.8 (basic.life) paragraph 9:

Creating a new object at the storage location that a `const` **complete** object with static, ~~or thread, or automatic~~ storage duration occupies or, at the storage location that such a `const` object used to occupy before its lifetime ended results in undefined behavior. [Example: ...]

This is necessary to allow types such as `std::optional` to contain `const` subobjects; the existing restriction exists to allow ROMability, and so only affects complete objects and is not necessary for automatic storage duration objects.

Change in 3.9.2 (basic.compound) paragraph 3:

[...] Every value of pointer type is one of the following:

- **a pointer to an object or function (the pointer is said to *point to* the object or function), or**
- **a pointer past the end of an object (5.3.1 `expr.unary.op`), or**
- **the null pointer value (4.10 `conv.ptr`) for that type, or**
- **an invalid pointer value.**

A valid value of an object pointer type **that is a pointer to or past the end of an object** ~~represents either the address~~ **represents the address** of **a the first** byte in memory (1.7 `intro.memory`) **that the object occupies or the first byte in memory after the end of the storage occupied by the object, respectively** or a null pointer (4.10). If an object of type `T` is located at an address `A`, a pointer of type `cv T*` whose value is the address `A` is said to point to that object, regardless of how the value was obtained. [Note: For instance, the address one **A pointer** past the end of an array object (5.7) ~~would be~~ **is not** considered to point to an unrelated object of the array's element object's type that might be located at that address. ~~There are further restrictions on pointers to objects with dynamic storage duration; see 3.7.4.3.~~ **A pointer acquires an invalid pointer value when the storage it denotes reaches the end of its storage duration; see [basic.stc]. — end note]** For purposes of pointer arithmetic (5.7 `expr.add`) and comparison (5.9 `expr.rel`, 5.10 `expr.eq`), a pointer that points past the end of the last element of an array `x` of `n` elements is considered to point to a hypothetical element `x[n]`. The value representation of pointer types is implementation-defined. Pointers to `cv`-qualified and `cv`-unqualified versions (3.9.3) of layout-compatible types shall have the same value representation and alignment requirements (3.11). [Note: ... — end note]

Change in 5.7 (`expr.add`) paragraph 4:

When an expression that has integral type is added to or subtracted from a pointer, the result has the type of the pointer operand. ~~If the pointer operand points to an element of an array object [Footnote], and the array is large enough, the result points to an element offset from the original element such that the difference of the subscripts of the resulting and original array elements equals the integral expression. In other words, if~~ **If the expression `P` points to the i -th element `x[i]` of an array object `x` with n elements [Footnote: An object that is not an array element is considered to belong to a single-element array for this purpose; see 5.3.1. A pointer past the end of the last**

element of an array x of n elements is considered to point to a hypothetical element $x[n]$ for this purpose; see 3.9.2.], the expressions $(P)+N$ (equivalently, $N+(P)$) and $(P)-N$ $P + J$ and $J + P$ (where N has the value n) point to, respectively, the $x[i + j]$ -th and $x[i - j]$ -th elements of the array object, provided they exist if $0 \leq i + j < n$, and point past the end of $x[n - 1]$ if $i + j = n$. Moreover, if the expression P points to the last element of an array object, the expression $(P)+1$ points one past the last element of the array object, and if the expression Q points one past the last element of an array object, the expression $(Q)-1$ points to the last element of the array object. If both the pointer operand and the result point to elements of the same array object, or one past the last element of the array object, the evaluation shall not produce an overflow; otherwise, the behavior is undefined. Likewise, the expression $P - J$ points to $x[i - j]$ if $0 \leq i - j < n$, and point past the end of $x[n - 1]$ if $i - j = n$; otherwise, the behavior is undefined.

Change in 5.7 (expr.add) paragraph 5:

When two pointers to elements of the same array object are subtracted, the result is the difference of the subscripts of the two array elements. The type of the result is an implementation-defined signed integral type; this type shall be the same type that is defined as `std::ptrdiff_t` in the `<cstdlib>` header (18.2). As with any other arithmetic overflow, if the result does not fit in the space provided, the behavior is undefined. In other words, if the expressions P and Q point to, respectively, the i -th and j -th elements $x[i]$ and $x[j]$ of an array object x , the expression $(P)-(Q)$ has the value $i - j$ provided the value fits in an object of type `std::ptrdiff_t`; otherwise, the behavior is undefined. [Note: If the value $i - j$ is not in the range of representable values of type `std::ptrdiff_t`, the behavior is undefined. — end note] Moreover, if the expression P points either to an element of an array object or one past the last element of an array object, and the expression Q points to the last element of the same array object, the expression $((Q)+1)-(P)$ has the same value as $((Q)-(P))+1$ and as $-(P)-((Q)+1)$, and has the value zero if the expression P points one past the last element of the array object, even though the expression $(Q)+1$ does not point to an element of the array object. Unless both pointers point to elements of the same array object, or one past the last element of the array object, the behavior is undefined. [Footnote: Another way to approach pointer arithmetic ...]

Change in 5.7 (expr.add) paragraph 7:

If the value 0 is added to or subtracted from a null pointer value, the result compares equal to the original pointer value is a null pointer value. If two pointers point to the same object or both point past the end of the same array or both are null, and the two pointer values are subtracted, the result compares equal to the value 0 converted to the type `std::ptrdiff_t`.

Change in 5.9 (expr.rel) paragraph 3:

Comparing unequal pointers to objects [Footnote: An object that is not an array element is considered to belong to a single-element array for this purpose; see 5.3.1. A pointer past the end of the last element of an array of n elements is considered to point to a hypothetical element n for this purpose; see 3.9.2.] is defined as follows:

- If two pointers point to different elements of the same array, or to subobjects thereof, the pointer to the element with the higher subscript compares greater.
- If one pointer points to an element of an array, or to a subobject thereof, and another pointer points one past the last element of the array, the latter pointer compares greater.
- If two pointers point to different non-static data members of the same object, or to subobjects of such members, recursively, the pointer to the later declared member compares greater provided the two members have the same access control (Clause 11) and provided their class is not a union.

Drafting note: the change in 3.9.2 affects the semantics of an example like:

```
struct S {
    char a;
    int b;
} s;
static_assert(static_cast<void*>(&s.b) >= static_cast<void*>(&s.a + 1));
```

Prior to this change, the result of the comparison was unspecified; now, it is required to hold, because `&s.a + 1` is treated as a pointer to an element of the "array" `s.a`. The value of `&s.b > &s.a + 1` is now specified and is `false` if and only if the pointers represent the same address.

Add a new paragraph after 5.18 (expr.ass) paragraph 6:

When the left operand of an assignment operator involves a member access expression (5.2.5 [expr.ref]) that nominates a union member, it may begin the lifetime of that union member, as described below. For an expression

E, define the set $S(E)$ of subexpressions of E as follows:

- If E is of the form $A.B$, $S(E)$ contains the elements of $S(A)$, and also contains $A.B$ if B names a union member of a non-class, non-array type, or of a class type with a trivial default constructor that is not deleted, or an array of such types.
- If E is of the form $A[B]$ and is interpreted as a built-in array subscripting operator, $S(E)$ is $S(A)$ if A is of array type and is $S(B)$ otherwise.
- Otherwise, $S(E)$ is empty.

In an assignment expression of the form $E_1 = E_2$, for each element E_i of $S(E_1)$, if modification of E_i would have undefined behavior under 3.8 [basic.life], an object of the type of E_i is implicitly created with vacuous initialization in the nominated storage; the beginning of its lifetime is sequenced after the value computation of the left and right operands and before the assignment. [Note:

This ends the lifetime of the previously-active member of the union, if any (3.8 [basic.life], 9.5 [class.union]).

— end note]

[Example:

```
union A { int x; int y[4]; };
struct B { A a; };
union C { B b; int k; };
int f() {
    C c; // does not start lifetime of any union member
    c.b.a.y[3] = 4; // creates objects to hold union members c.b and c.b.a.y
    return c.b.a.y[3]; // OK: c.b.a.y refers to newly created object (see 3.8 [basic.life])
}

struct X { const int a; int b; };
union Y { X x; int k; };
void g() {
    Y y = { { 1, 2 } }; // OK, y.x is active union member (9.2)
    int n = y.x.a;
    y.k = 4; // OK: ends lifetime of y.x, y.k is active member of union
    y.x.b = n; // undefined behavior: y.x.b modified outside its lifetime,
              // union member y.x's lifetime does not start because X's
              // default constructor is deleted
}

— end example ]
```

Add a new paragraph before 8.3.4 (dcl.array) paragraph 7:

The address of an array object is the same as the address of its first element. [Note: Nonetheless, if a pointer to the object is cast to a pointer to the type of its first subobject (or vice versa), the resulting pointer does not point to the subobject and cannot be used to access it. —end note]

Change in 9.2 class.mem paragraph 19:

In a standard-layout union with an active member (9.5) of struct type T_1 , it is permitted to read a non-static data member m of another union member of struct type T_2 provided m is part of the common initial sequence of T_1 and T_2 ; **the behavior is as if the corresponding member of T_1 were nominated.**

Change in 9.2 class.mem paragraph 20:

If a standard-layout class object has any non-static data members, its address is the same as the address of its first non-static data member. Otherwise, its address is the same as the address of its first base class subobject (if any). [Note: ...—end note] **[Note: Nonetheless, if a pointer to the object is cast to a pointer to the type of its first subobject (or vice versa), the resulting pointer does not point to the subobject and cannot be used to access it. —end note]**

Change in 9.5 class.union paragraph 1, splitting it into two paragraphs:

In a union, **a non-static data member is active if its name refers to an object whose lifetime has begun and has not ended ([basic.life]).** **At** most one of the non-static data members **of an object of union type** can be active at any time, that is, the value of at most one of the non-static data members can be stored in a union at any time. [Note: One special guarantee is made in order to simplify the use of unions: If a standard-layout union contains several standard-layout structs that share a common initial sequence (9.2), and if **the active member of** an object

of this standard-layout union type **contains** **is** one of the standard-layout structs, it is permitted to inspect the common initial sequence of any of **the** standard-layout struct members; see 9.2. — *end note*]

The size of a union is sufficient to contain the largest of its non-static data members. Each non-static data member is allocated as if it were the sole member of a struct. All non-static data members of a union object have the same address. **[Note: Nonetheless, if a pointer to a union object is cast to a pointer to the type of a non-static data member (or vice versa), the resulting pointer does not point to the subobject and cannot be used to access it. —end note]**

Add to 18.6 (support.dynamic) paragraph 1:

```
namespace std {  
    template <class T>  
        constexpr T* launder(T* p) noexcept;  
}
```

Add a new subclause under 18.6 (support.dynamic):

```
template <class T> constexpr T* launder(T* p) noexcept;
```

Requires: p represents the address A of a byte in memory, and an object x that is within its lifetime and whose cv-qualified type is similar to T is located at the address A .

Returns: A value of type T * that points to x .

Note: An invocation of this function may be used in a core constant expression whenever the value of its argument may be used in a core constant expression.

[Example:

```
struct X { const int n; };  
X *p = new X{3};  
const int a = p->n;  
new (p) X{5}; // p does not point to new object ([basic.life])  
// because X::n is const  
const int b = p->n; // undefined behavior  
const int c = std::launder(p)->n; // ok
```

— end example]

Drafting note: see N4303 for more background on the purpose and intended usage of this function.

Change in 20.7.5 (ptr.align) paragraph 1:

Effects: If it is possible to fit $size$ bytes of storage aligned by $alignment$ into the buffer pointed to by ptr with length $space$, the function updates ptr to **point to** **represent** the first possible address of such storage and decreases $space$ by the number of bytes used for alignment.

Change in 20.7.5 (ptr.align) paragraph 2:

Requires:

- $alignment$ shall be a power of two
- ptr shall **point to** **represent the address of** contiguous storage of at least $space$ bytes